

JavaScript Study Notes

A Smarter Way to Learn JavaScript — Chapters 10 to 17

Topics covered today: if statements, comparison operators, if/else/else-if, combining conditions with && and ||, nested if statements, and arrays (creating, adding, removing, slicing, splicing).

1. The if Statement (Ch. 10)

An **if statement** runs code only when a condition is true. Structure: keyword **if**, a space, the condition in parentheses, then a block in curly braces.

```
var x = prompt("Where does the Pope live?");
if (x === "Vatican") {
  alert("Correct!");
}
```

Key rules:

- Use **===** (triple equals) to test a value, never a single **=** (that's assignment, not comparison).
- Any number of statements can run inside the block.
- Curly braces can technically be skipped for one-line ifs, but keep them for consistency and readability.

2. Comparison Operators (Ch. 11)

Operator	Meaning
===	equal to
!==	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

String comparisons are case-sensitive — **"Rose" !== "rose"**. **==** and **!=** also work but are looser (they allow type coercion); stick to **===** and **!==** for predictable results.

3. if...else and else if (Ch. 12)

```

var correctAnswer = "Vatican";
if (x === correctAnswer) {
    alert("Correct!");
}
else if (x === "Rome") {
    alert("Incorrect but close");
}
else {
    alert("Incorrect");
}

```

JavaScript stops checking as soon as one condition tests true — it never bothers checking the rest of the chain. This is the cleaner alternative to writing two separate if blocks (one for the true case, one for the false case).

4. Testing Sets of Conditions: && and || (Ch. 13)

- **&& (AND)** — every condition must be true.
- **|| (OR)** — at least one condition must be true.

```

if (weight > 300 && time < 6) {
    alert("Come to our tryout!");
}

if (SAT > avg || GPA > 2.5 || sport === "football") {
    alert("Welcome to Bubba State!");
}

```

Watch out for ambiguity:

When && and || are mixed without parentheses, the meaning can be read two different ways. Always use parentheses to force the grouping you actually intend:

```

if ((age > 65 || age < 21) && res === "U.S.") { ... } // one meaning
if (age > 65 || (age < 21 && res === "U.S.)) { ... } // a different meaning

```

5. Nested if Statements (Ch. 14)

An if can contain other if statements inside it. Nest levels are shown by the position of the curly braces — indent each deeper level by 2 spaces for readability. For simple combinations, combining conditions with && / || (Ch. 13) is often cleaner than nesting; nesting earns its keep when the logic gets genuinely complex.

```
if (c === d) {  
  if (x === y) {  
    g = h;  
  } else if (a === b) {  
    g = h;  
  } else {  
    e = f;  
  }  
} else {  
  e = f;  
}
```

6. Arrays (Ch. 15)

An array holds multiple values in one variable. Elements are indexed automatically, starting at 0.

```
var cities = ["Atlanta", "Baltimore", "Chicago", "Denver"];  
alert("Welcome to " + cities[3]); // "Welcome to Denver"
```

- First element is always index 0, not 1.
- An array can mix types: numbers, strings, booleans in the same array.
- Declare with **var** once; drop **var** when reassigning later.

7. Arrays — Adding, Removing, Extracting (Ch. 16–17)

Method	What it does
push(x)	adds x to the end
pop()	removes the last element
unshift(x)	adds x to the beginning
shift()	removes the first element
splice(i, n, ...x)	at index i, remove n elements and insert x (either or both optional)
slice(i, j)	copies elements from index i up to (not including) j into a new array

slice's second number is the index *after* the last element you want — it's easy to misread it as the last included index. splice's first number is the start index and the second is how many existing elements to delete from there.

8. Review of Today's Practice (app.js)

You put all of this to work in your own file. Here's honest feedback on what's there.

What you did well

- Correct use of `===` for both boolean and string/number comparisons, including proving `2 === "2"` is false (different types).
- Good technique using `+ prompt(...)` to convert the prompt's string return value into a number in one step, before comparing it with `>=` and `<`.
- The grading nested-if (checking 0–100, then A+/A/B/C/F bands) is logically correct and the boundaries don't overlap.
- All six array methods from today — `pop`, `push`, `unshift`, `shift`, `slice`, `splice` — are used correctly and each block correctly re-declares the starting array before demonstrating the method.

One thing to tighten up

Your grading logic nests three levels deep (range check, then range check, then the grade bands). Chapter 12 makes the case that when you're really just testing "which one of several conditions is true," a flat else-if chain reads more clearly than nesting. You could flatten the grade-band part like this, keeping only the outer range check nested:

```
if (studentPercentage < 0) {
  console.log("Number is less than 0");
} else if (studentPercentage > 100) {
  console.log("Number is greater than 100");
} else if (studentPercentage >= 80) {
  console.log("A+");
} else if (studentPercentage >= 70) {
  console.log("A");
} else if (studentPercentage >= 60) {
  console.log("B");
} else if (studentPercentage >= 40) {
  console.log("C");
} else {
  console.log("F");
}
```

Same result, one level of nesting instead of three — easier to scan and easier to add a grade band to later. (Also: it's "greater than," not "greater then" — a small typo in the two `console.log` strings, worth fixing since real output will show it.)

Nothing else to correct — the rest of the file is clean, and the array section shows you've actually internalized the difference between mutating methods (`pop`, `push`, `unshift`, `shift`, `splice`) and `slice`, which copies without touching the original.